

Lab 01: Building and Exploring a Discrete Environment

Objective

Build a `DiscreteEnvironment` from scratch, step through it, record trajectories, and visualize the generative process.

Prerequisites

- Python with NumPy and matplotlib installed
- Familiarity with probability distributions and matrix notation
- Access to the `active_inference` library (`src/active_inference/`)

Part 1: Constructing the Environment

Goal: Create a 3-state, 2-observation environment with 2 actions.

1. Define a `true_A` matrix of shape `(2, 3)` where:
2. State 0 produces observation 0 with probability 0.8
3. State 1 produces each observation with equal probability
4. State 2 produces observation 1 with probability 0.9
5. Define a `true_B` tensor of shape `(3, 3, 2)` where:
6. Action 0 is the identity (stay in current state)
7. Action 1 rotates: state $0 \rightarrow 1$, state $1 \rightarrow 2$, state $2 \rightarrow 0$
8. Create a `DiscreteEnvironment` starting in state 0.

```
import numpy as np
from active_inference.agent import DiscreteEnvironment

# TODO: Define true_A and true_B
# TODO: Create environment
```

Response:

Write your response here

Part 2: Running a Trajectory

Goal: Step the environment for 20 timesteps using a fixed action sequence.

1. Reset the environment to state 0.
2. Alternate between action 0 (stay) and action 1 (rotate) for 20 steps.
3. Record the state, observation, and action at each step.
4. Print the full trajectory.

```
env.reset(initial_state=0)
for t in range(20):
    action = t % 2 # alternate stay/rotate
    obs = env.step(action)
    print(f"t={t}: action={action}, state={env.state}, obs={obs}")
```

Response:

Write your response here

Part 3: Visualizing the Trajectory

Goal: Use `plot_environment_trajectory` to visualize the state-observation-action sequence.

1. After running Part 2, extract `env.history`.
2. Call `plot_environment_trajectory` to generate a figure.
3. Save the figure to `output/lab01_trajectory.png`.

```
from active_inference.visualization import plot_environment_trajectory

plot_environment_trajectory(
    states=env.history["states"],
    observations=env.history["observations"],
    save_path="output/lab01_trajectory.png",
)
```

Response:

Write your response here

Part 4: Empirical Likelihood Estimation

Goal: Verify the `true_A` matrix by collecting observations.

1. Reset the environment. Fix the state by using action 0 (identity) from a known starting state.
2. Collect 1000 observations from each state (reset to each state, step with identity action 1000 times).
3. Compute the empirical frequency of each observation per state.
4. Compare with the original `true_A` matrix.

Response:

Write your response here

Part 5: Analysis Questions

1. When you ran 20 steps with alternating actions, how many unique states did the trajectory visit? Was this predictable from the `true_B` matrix?
2. Did your empirical likelihood estimates in Part 4 converge to the true values? How many samples were needed for 2-decimal accuracy?
3. What would change if you made `true_A = np.eye(3, 2).T` (a non-square identity-like matrix)? Would the environment still be valid?

Response:

Write your response here

Summary

Skill	Library Component	Status
Create environments with custom A and B matrices	<code>DiscreteEnvironment(true_A, true_B)</code>	
Step through environments and collect trajectories	<code>env.step(action)</code> , <code>env.history</code>	
Visualize state-observation trajectories	<code>plot_environment_trajectory()</code>	
Empirically verify likelihood matrices	Manual frequency counting vs <code>true_A</code>	
Understand the generative process abstraction	Generative process $\neq$ generative model	